

HW-3: State Management

Author: Arjun Taneja

1. States on the Android Platform

The main states that an android application can enter are:

- **Active/Running:** The app is in the foreground and actively being used by the user. It has focus and is receiving user input.
- **Paused:** The app is partially visible but doesn't have focus. This can happen when another activity or dialog appears on top of the app, such as an incoming phone call, a system overlay, or a multi-window mode. In the paused state, the app is still alive and maintains its state, but it may not be actively executing code. I extensively used the multi-window functionality offered on the Xiaomi Poco F1. I really wish the Google Pixel phones allowed the user to operate multiple apps in split-screen too.
- **Stopped:** The app is completely hidden and not visible to the user. It is still present in memory, but the system may kill the app's process if it needs to free up resources. When the user navigates back to the app, it may need to be restarted and restored to its previous state. E.g. Clash of Clans always reloads itself, even when it loses focus for a few seconds.
- **Killed/Destroyed:** The app process is terminated by the system, either due to resource constraints or by the user manually closing the app. When the app is launched again, it needs to start from scratch and restore its state if necessary.

2. States to consider for JamBook

For JamBook, I'm considering the following states:

- **Active/Running:**
 - When the app is in the active state, users should be able to interact with the UI, navigate between screens, make reservations, and view their existing reservations.
 - The app should respond to user input and update the UI accordingly.
 - If the app is fetching data from an API or performing any background tasks, it should handle them efficiently without blocking the UI.

- **Paused:**

- When the app is paused, it should save any necessary data and state information.
- If the user is in the middle of making a reservation, the app should store the selected date and any other relevant details.
- If the user is viewing their reservations, the app should remember the scroll position or any filters applied.
- The app should release any resources that are not needed while in the paused state to optimize memory usage.

- **Stopped:**

- When the app is stopped, it should save any critical data and state information to persistent storage (e.g., database, shared preferences) so that it can be restored later if needed.
- The app should gracefully handle the situation where the system kills the app process while in the stopped state. It should be able to recover and restore the user's previous state when the app is launched again.

- **Killed/Destroyed:**

- If the app is killed or destroyed by the system, it should have mechanisms in place to restore the user's data and state when the app is launched again.
 - The app should handle any necessary data migrations or updates that may be required if the app's data schema has changed between versions.
-